

DOCKER

IMAGES

```
docker pull nginx # Download image
docker images # List local images
docker build -t myapp:1.0 . # Build from Dockerfile
docker rmi <image> # Remove image
docker image prune # Remove unused images
docker search ubuntu # Search Docker Hub
```

CONTAINERS — RUN

```
docker run -d -p 80:80 --name web nginx
docker run -it ubuntu bash # Interactive
docker run --rm nginx # Auto-remove on exit
docker run -e KEY=val nginx # Env variable
docker run -v myvol:/data nginx # Mount volume
```

CONTAINERS — MANAGE

```
docker ps # Running containers
docker ps -a # All containers
docker stop <id> # Stop gracefully
docker start <id> # Start stopped
docker restart <id>
docker rm <id> # Remove container
docker rm -f <id> # Force remove
docker exec -it <id> bash # Shell into container
docker logs <id> # View logs
docker logs -f <id> # Follow live logs
docker inspect <id> # Full JSON details
docker stats # Live CPU/memory
docker cp <id>:/app ./local # Copy files out
```

VOLUMES & NETWORKS

```
docker volume create myvol # Create volume
docker volume ls # List volumes
docker network create mynet # Create network
docker network ls # List networks
docker network inspect mynet
```

DOCKER RUN FLAGS

Flag	Meaning
-d	Detached (background)
-it	Interactive + TTY
-p 80:80	host:container port
-v vol:/path	Mount volume
--name web	Name the container
--rm	Auto-remove on stop
-e KEY=val	Set env variable
--network	Connect to network

DOCKER SWARM

INIT & JOIN

```
# --- MANAGER ---
docker swarm init --advertise-addr <IP>
docker node ls # See all nodes
docker swarm leave --force

# --- WORKER ---
docker swarm join --token SWMTKN-xxx <IP>:2377
```

SERVICES

```
docker service create --name web --replicas 2 -p 80:80 nginx
docker service ls # List services
docker service ps web # Tasks / replicas
docker service scale web=4
docker service update --image nginx:new web
docker service logs web
docker service rm web
```

NODE & STACK

```
docker node inspect <node>
docker node update --availability drain <node>
docker stack deploy -c docker-compose.yml mystack
docker stack ls # List stacks
docker stack rm mystack
```

SWARM PORTS

Port	Purpose
2377	Cluster management (TCP)
7946	Node communication (TCP/UDP)
4789	Overlay network (UDP)

DOCKER COMPOSE

DOCKER-COMPOSE.YML

```
version: "3"
services:
  web:
    image: nginx
    ports: ["80:80"]
    depends_on: [db]
    environment:
      - APP_ENV=prod
    volumes: ["/html:/usr/share/nginx/html"]
  db:
    image: postgres:15
    environment:
      POSTGRES_PASSWORD: secret
    volumes: ["pgdata:/var/lib/postgresql/data"]
volumes:
  pgdata:
```

COMPOSE COMMANDS

```
docker-compose up -d # Start detached
docker-compose down # Stop & remove
docker-compose down -v # Also remove volumes
docker-compose ps # Status
docker-compose logs -f web # Follow service logs
docker-compose build # Rebuild images
docker-compose exec web bash # Shell into service
docker-compose restart
docker-compose stop / start
```

KIND — CLUSTER COMMANDS

```
kind create cluster # Single-node
kind create cluster --config kind-config.yaml --name mycluster
kind get clusters # List clusters
kind delete cluster --name mycluster
```

KIND-CONFIG.YAML

```
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
  - role: control-plane
  - role: worker
```

BASH SCRIPTING

BASICS

```
#!/bin/bash # Shebang – always first!
read -p "Enter: " var # Read with
prompt
read -n 10 var # Read exactly N chars
echo ${var} # Length of string
echo ${var:0:3} # Substring (start,
len)
printf '%d' "'A" # ASCII value of char
```

VARIABLES & ARITHMETIC

```
name="Ali" # Assign variable
echo $name # Use variable
x=$((5 + 3)) # Arithmetic
x=$((x + 1)) # Increment
result=$((command)) # Store output
$0 $1 $2 # Script name, arg1, arg2
 $? # Last exit code (0=success)
```

CONDITIONALS

```
if [ "$x" -eq 5 ]; then # equal
    echo "yes"
elif [ "$x" -gt 5 ]; then # greater
    echo "greater"
else
    echo "less"
fi

if [[ "$s" = "hi" ]]; then ...; fi #
String
if [[ "$c" =~ [aeiou] ]]; then ...; fi
# Regex

[ -f file ] # file exists | [ -d dir
] # dir exists
```

LOOPS

```
for i in 1 2 3; do # For in list
    echo $i
done

for ((i=0; i<10; i++)); do # C-style
    for
        echo $i
    done
done

while [ "$x" -lt 10 ]; do # While loop
    x=$((x+1))
done
```

FUNCTIONS & I/O

```
greet() {
    echo "Hello $1" # $1 = first arg
}

greet "World" # Call function

echo "text" > file.txt # Write
(overwrite)
echo "text" >> file.txt # Append

cmd1 | cmd2 # Pipe output

cmd1 && cmd2 # Run if first succeeds
cmd1 | cmd2 # Run if first fails
```

TEST OPERATORS

Operator	Meaning
<code>-eq</code>	Equal (numbers)
<code>-ne</code>	Not equal
<code>-gt</code>	Greater than
<code>-lt</code>	Less than
<code>-ge / -le</code>	Greater/less or equal
<code>-z</code>	String is empty
<code>-n</code>	String is not empty
<code>-f</code>	File exists
<code>-d</code>	Directory exists